

MODULE 4 · NETWORKING

Ingress

One entry point, host & path rules fan out to Services

WHY IT MATTERS

One external door beats one per Service

A NodePort or LoadBalancer Service exposes **one** backend. Ten HTTP apps means ten node ports (or ten external IPs) to track, firewall, and remember.

Ingress is an L7 HTTP(S) router: one entry point that reads the **Host header and URL path** and fans requests out to the right Service.

BY THE END YOU CAN

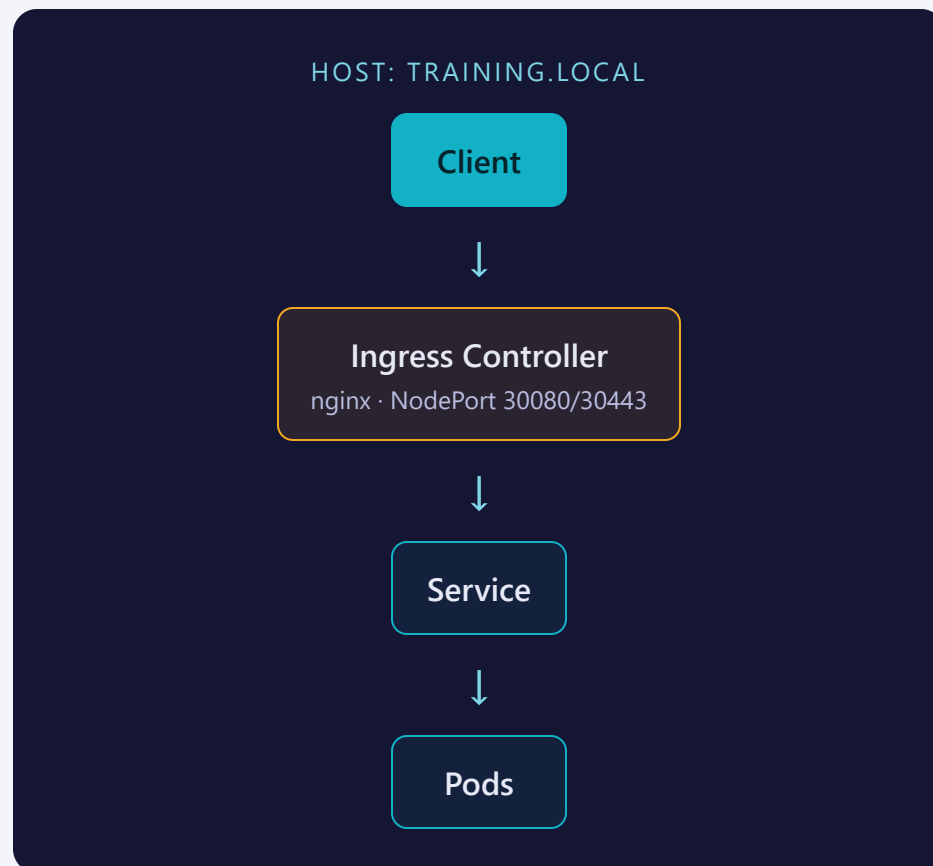
- Explain why Ingress needs a controller
- Write host/path rules that fan out to Services
- Trace a request through rewrite-target
- Diagnose a missing-Host-header 404

CONCEPT

Ingress is a rulebook — a controller reads it

The Ingress object itself does nothing. It needs an **Ingress Controller** — a Pod running proxy software that watches Ingress objects and programs itself to match.

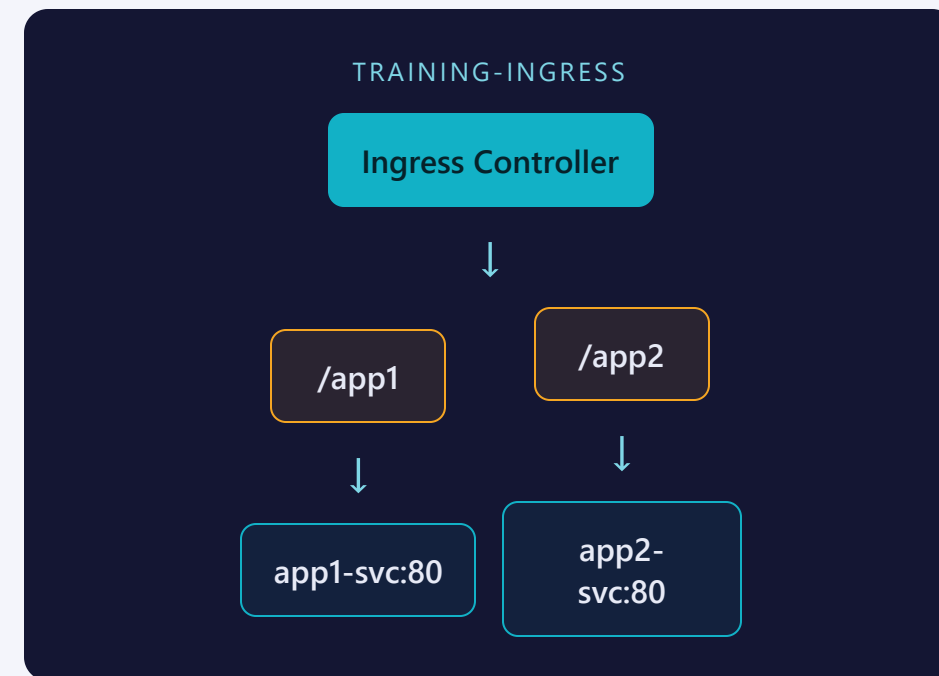
Here that controller is **nginx**, reached on the node's NodePort 30080 (HTTP) / 30443 (HTTPS) — no cloud load-balancer involved.



One controller, many rules, many backends

Each rule matches a **path** under a **host** and points at one Service.
pathType: Prefix means /app1 also matches /app1/anything.

The controller strips the matched prefix before forwarding when
rewrite-target: / is set — otherwise the backend sees a path it
doesn't have.



Rules map host + path to a Service

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: training-ingress
  namespace: training
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: training.local
      http:
        paths:
          - path: /app1
            pathType: Prefix
            backend:
              service:
                name: app1-svc
                port:
                  number: 80
          - path: /app2
            pathType: Prefix
            backend:
              service:
                name: app2-svc
                port:
                  number: 80
```

One rule fast, or the whole set declared

IMPERATIVE

```
kubectl create ingress training-ingress \  
--rule="training.local/app1=app1-svc:80"
```

DECLARATIVE

```
kubectl apply -f ingress.yaml
```

THE BRIDGE TRICK

```
$ kubectl create ingress training-ingress \  
--rule="training.local/app1=app1-svc:80" --dry-run=client -o yaml > ingress.yaml
```

SEE IT

The Host header decides the route

Same node port, same path, different Host header — completely different result. That's the whole trick behind fan-out on one controller.

No Host header at all → nginx has no rule to match → 404, not a connection error.

```
$ curl -H 'Host: training.local' NODE_IP:30080/app1
# nginx welcome page

$ curl -H 'Host: training.local' NODE_IP:30080/app2
# It works! (httpd)

$ curl NODE_IP:30080/app1
# 404 — no Host header matched
```

One controller, one pair of ports, N routes

	N SERVICES, N FRONT DOORS	N SERVICES, ONE INGRESS
External surface	N node ports / N external IPs	1 controller · NodePort 30080/30443
TLS termination	configured N times	configured once, per host
Add a service	new port/IP, new firewall rule	add a rule to the same Ingress

THE SHIFT

L4 Services scale the number of doors with the number of apps. Ingress keeps the door count fixed and scales the **rulebook** instead.

Where people trip

- **No controller running.** The Ingress object accepts fine but nothing ever answers on the port.
- **Missing Host header.** Hitting the node port without -H 'Host: ...' looks like the Ingress is broken — it isn't.
- **Forgetting rewrite-target.** Backend gets the full /app1 path and 404s inside the app itself.
- **ingressClassName mismatch.** Rule sits unmatched if it doesn't name the controller that's actually running.

Commands to keep at hand

```
$ kubectl get ingress -n training           # hosts, address, ports
$ kubectl describe ingress NAME -n training # rules + backends
$ curl -H 'Host: HOST' NODE_IP:30080/PATH   # exercise a rule
$ kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx # controller's view
```

Tip: when a route "doesn't work", check `describe ingress` before the app — a missing or unmatched rule never reaches the Pod.

RECAP

Ingress in one breath

- Ingress = L7 HTTP(S) rulebook; needs a controller (nginx here) to act on it
- Rules match host + path, each pointing at a Service
- `rewrite-target` strips the matched prefix before forwarding
- One controller + N rules replaces N node ports / N LoadBalancers

HANDS-ON

Now run Lab 4.5
`labs/4.5/`

Next: [4.6 — TLS Ingress](#)